K. R. MANGALAM UNIVERSITY

B. TECH CSE (Data Science)

SOET

(School of Engineering &Technology)

ENCS256-ANALYSIS AND DESIGN OF ALGORITHMS

Submitted by-

Name - Sakshi Jain

Roll No – 2401420034

Submitted to-

Aarti Ma'am

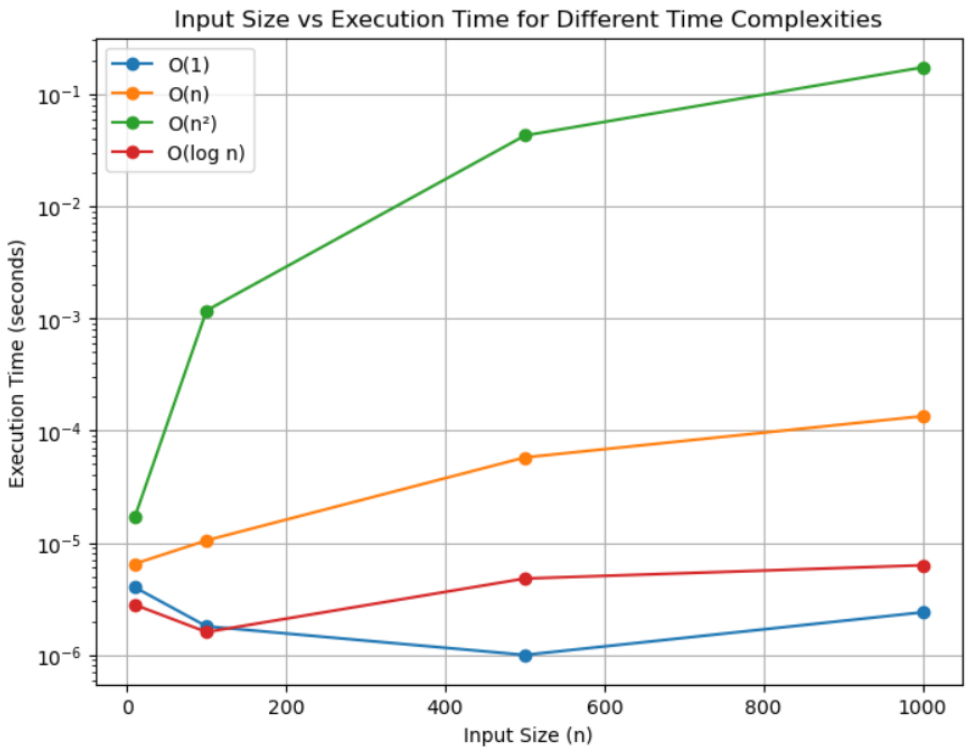
TASK 1: ALGORITHM GROWTH OBSERVATION

Code :

```
import time
import matplotlib.pyplot as plt

# ---- Algorithms ----
def constant_time(n):
    x=n*n
def linear_time(n):
    s=0
    for i in range(n):
        s+=i
def quadratic_time(n):
    s=0
    for i in range(n):
        for j in range(n):
            s+=i+j
def logarithmic_time(n):
    i=1
    while i<n:
        i*=2

# Input sizes
n_values=[10, 100, 500, 1000]
o1_times=[]
on_times=[]
on2_times=[]
ologn_times=[]
```

```python
# ---- Measure Execution Time ----
for n in n_values:
    start=time.perf_counter()
    constant_time(n)
    o1_times.append(time.perf_counter()-start)

    start=time.perf_counter()
    linear_time(n)
    on_times.append(time.perf_counter()-start)

    start = time.perf_counter()
    quadratic_time(n)
    on2_times.append(time.perf_counter() - start)

    start=time.perf_counter()
    logarithmic_time(n)
    ologn_times.append(time.perf_counter() - start)

# ---- Plot Graph ----
plt.figure(figsize=(8,6))
plt.plot(n_values,o1_times,marker='o',label="O(1)")
plt.plot(n_values,on_times,marker='o',label="O(n)")
plt.plot(n_values,on2_times,marker='o',label="O(n²)")
plt.plot(n_values,ologn_times,marker='o',label="O(log n)")
plt.xlabel("Input Size (n)")
plt.ylabel("Execution Time (seconds)")
plt.title("Input Size vs Execution Time for Different Time Complexities")

# IMPORTANT: Log scale makes small values visible
plt.yscale("log")
plt.legend()
plt.grid(True)
plt.show()
```

Output :


Input Size vs Execution Time for Different Time Complexities
O(1)
O(n)
O(n²)
O(log n)
Execution Time (seconds)
Input Size (n)
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{-5}
10^{-6}
0
200
400
600
800
1000

TASK 2: BEST, AVERAGE AND WORST CASE ALGORITHM

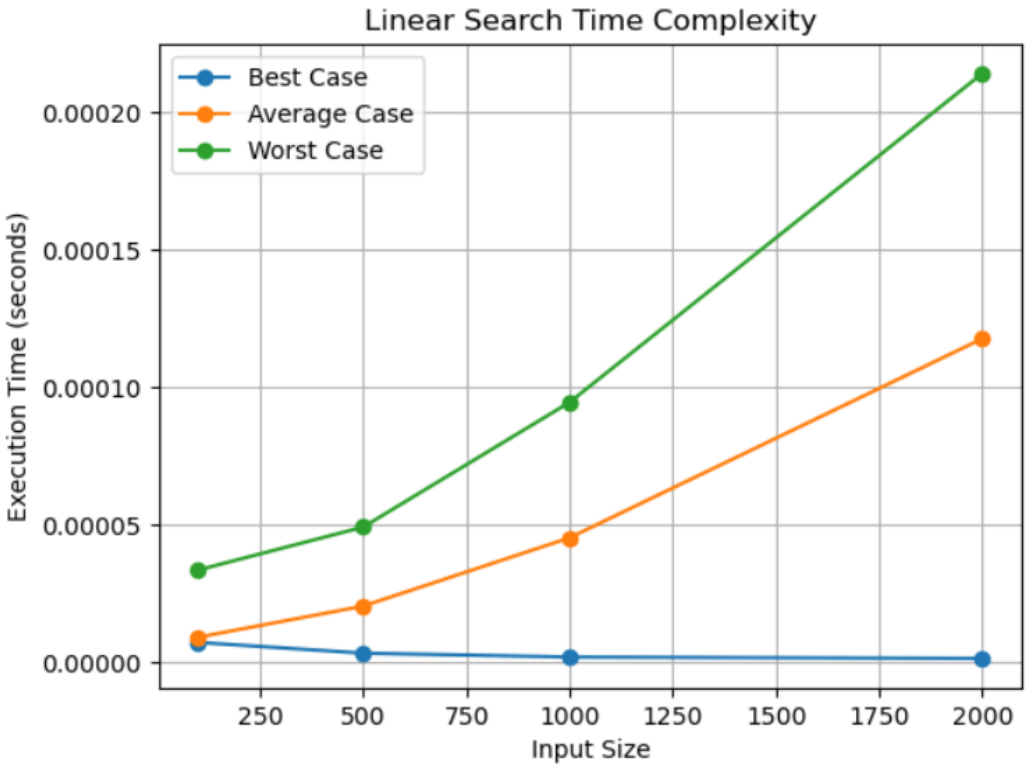
1. LINEAR SEARCH:

Code :

```
import time
import matplotlib.pyplot as plt

# Linear Search Function
def linear_search(arr,target):
    for i in range(len(arr)):
        if arr[i]==target:
            return i
    return -1

# Input sizes
sizes = [100,500,1000,2000]
best_case=[]
avg_case=[]
worst_case=[]
for n in sizes:
    arr=list(range(n))

    # Best Case (first element)
    start=time.perf_counter()
    linear_search(arr,arr[0])
    best_case.append(time.perf_counter()-start)

    # Average Case (middle element)
    start=time.perf_counter()
    linear_search(arr,arr[n//2])
    avg_case.append(time.perf_counter()-start)

    # Worst Case (last element)
    start=time.perf_counter()
    linear_search(arr,arr[-1])
    worst_case.append(time.perf_counter()-start)
```

```python
# Plot Graph
plt.plot(sizes,best_case,marker='o',label="Best Case")
plt.plot(sizes,avg_case,marker='o',label="Average Case")
plt.plot(sizes,worst_case,marker='o',label="Worst Case")
plt.title("Linear Search Time Complexity")
plt.xlabel("Input Size")
plt.ylabel("Execution Time (seconds)")
plt.legend()
plt.grid(True)
plt.show()
```

Output :



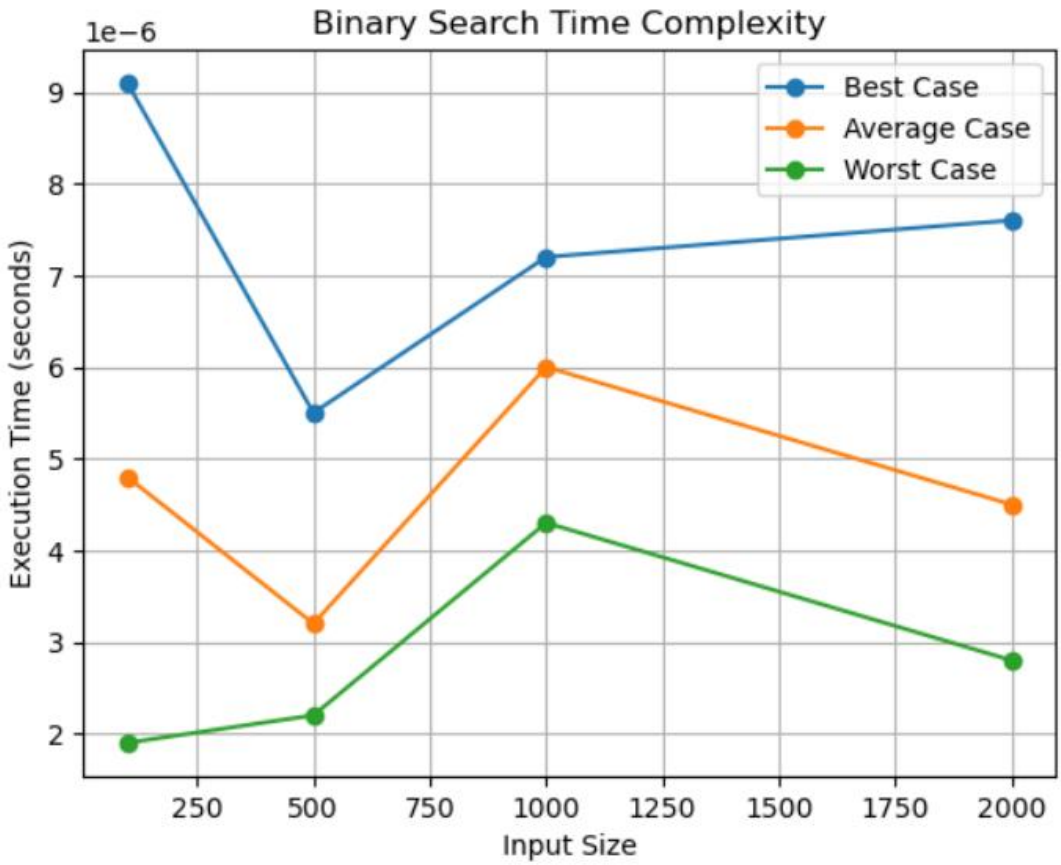
- **BINARY SEARCH:**

Code :

```
import time
import random
import matplotlib.pyplot as plt

# Binary Search Function
def binary_search(arr, target):
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==target:
            return mid
        elif arr[mid]<target:
            low=mid+1
        else:
            high=mid-1
    return -1

# Input sizes
sizes = [100,500,1000,2000]
best_case=[]
avg_case=[]
worst_case=[]
for n in sizes:
    arr=list(range(n))  # sorted array required

    # Best Case (middle element)
    start=time.perf_counter()
    binary_search(arr,arr[n//2])
    best_case.append(time.perf_counter()-start)
```

```
    # Average Case (random element)
    target=random.choice(arr)
    start=time.perf_counter()
    binary_search(arr,target)
    avg_case.append(time.perf_counter()-start)

    # Worst Case (element not present)
    start=time.perf_counter()
    binary_search(arr,-1)
    worst_case.append(time.perf_counter()-start)

# Plot Graph
plt.plot(sizes,best_case,marker='o',label="Best Case")
plt.plot(sizes,avg_case,marker='o',label="Average Case")
plt.plot(sizes,worst_case,marker='o',label="Worst Case")
plt.title("Binary Search Time Complexity")
plt.xlabel("Input Size")
plt.ylabel("Execution Time (seconds)")
plt.legend()
plt.grid(True)
plt.show()
```

Output :



TASK 3: RECURSION AND RECURRENCE VALIDATION

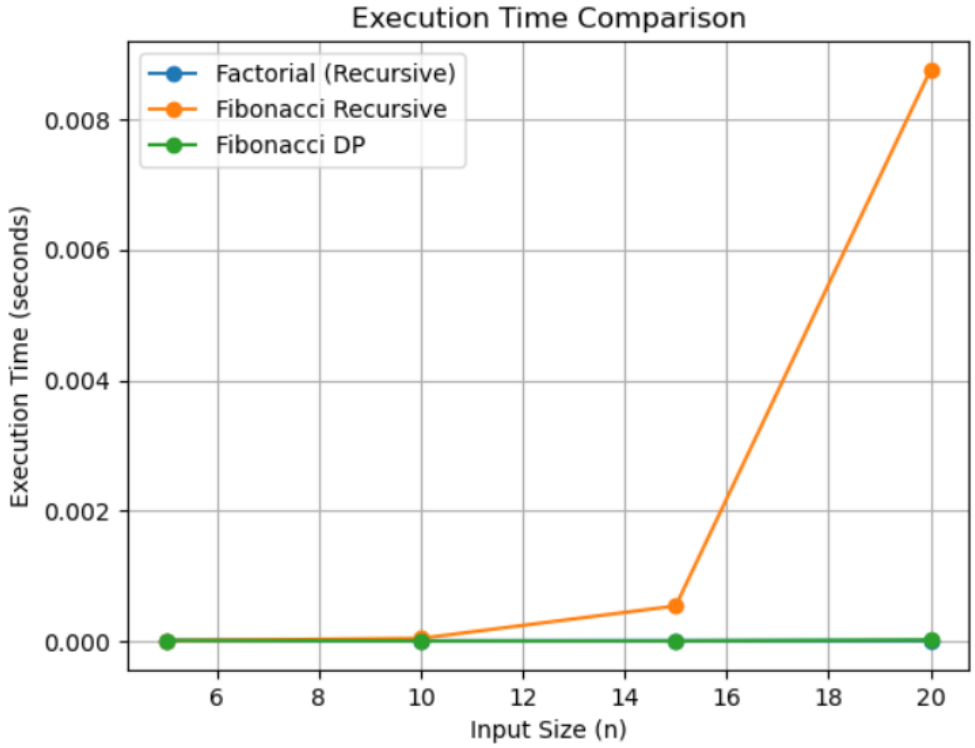
Code :

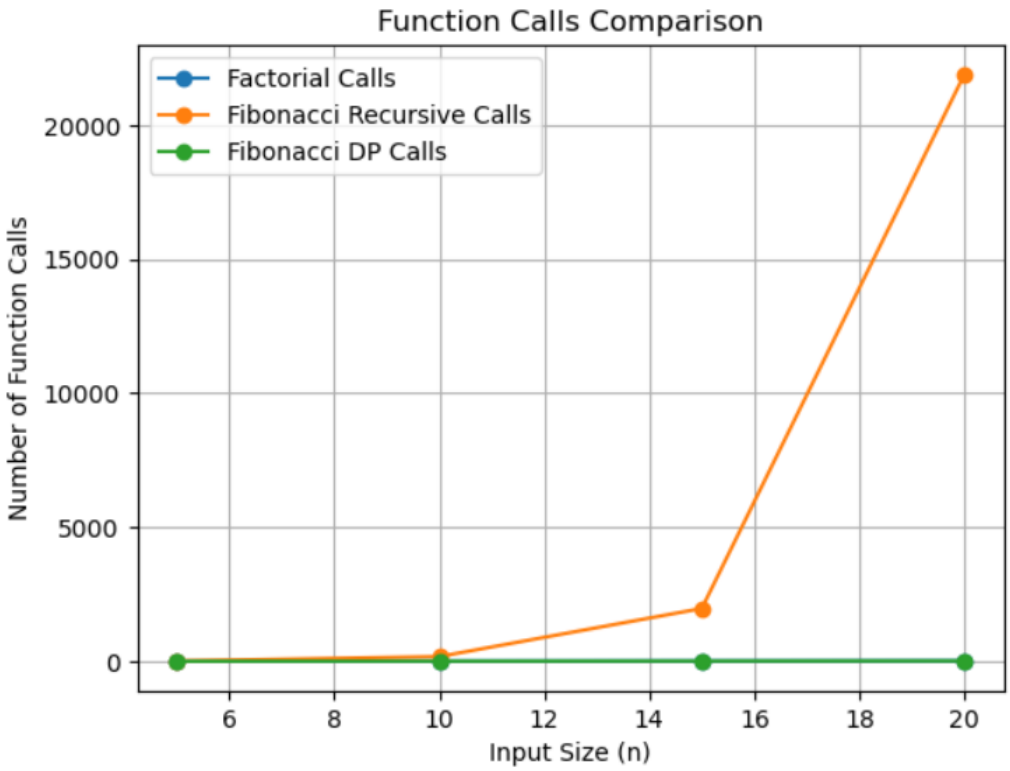
```
import time
import matplotlib.pyplot as plt

# Factorial
fact_calls=0
def factorial(n):
    global fact_calls
    fact_calls+=1
    if n==0 or n==1:
        return 1
    return n*factorial(n-1)

# Fibonacci Recursive
fib_calls=0
def fibonacci_recursive(n):
    global fib_calls
    fib_calls+=1
    if n<=1:
        return n
    return fibonacci_recursive(n-1)+fibonacci_recursive(n-2)

# Fibonacci Dynamic Programming
fib_dp_calls=0
def fibonacci_dp(n):
    global fib_dp_calls
    fib_dp_calls+=1
    dp=[0]*(n+1)
    if n>0:
        dp[1]=1
    for i in range(2,n+1):
        fib_dp_calls+=1
        dp[i]=dp[i-1]+dp[i-2]
    return dp[n]
```

```python
# Input sizes
n_values = [5,10,15,20]
fact_time=[]
fib_time=[]
fib_dp_time=[]
fact_call_list=[]
fib_call_list=[]
fib_dp_call_list=[]

for n in n_values:

    # Factorial
    fact_calls=0
    start=time.perf_counter()
    factorial(n)
    fact_time.append(time.perf_counter()-start)
    fact_call_list.append(fact_calls)

    # Fibonacci Recursive
    fib_calls=0
    start=time.perf_counter()
    fibonacci_recursive(n)
    fib_time.append(time.perf_counter()-start)
    fib_call_list.append(fib_calls)

    # Fibonacci DP
    fib_dp_calls=0
    start=time.perf_counter()
    fibonacci_dp(n)
    fib_dp_time.append(time.perf_counter()-start)
    fib_dp_call_list.append(fib_dp_calls)
```

```python
# Graph 1: Execution Time
plt.figure()
plt.plot(n_values,fact_time,marker='o',label="Factorial (Recursive)")
plt.plot(n_values,fib_time,marker='o',label="Fibonacci Recursive")
plt.plot(n_values,fib_dp_time,marker='o',label="Fibonacci DP")
plt.title("Execution Time Comparison")
plt.xlabel("Input Size (n)")
plt.ylabel("Execution Time (seconds)")
plt.legend()
plt.grid(True)
plt.show()

# Graph 2: Function Calls
plt.figure()
plt.plot(n_values,fact_call_list,marker='o',label="Factorial Calls")
plt.plot(n_values,fib_call_list,marker='o',label="Fibonacci Recursive Calls")
plt.plot(n_values,fib_dp_call_list,marker='o',label="Fibonacci DP Calls")
plt.title("Function Calls Comparison")
plt.xlabel("Input Size (n)")
plt.ylabel("Number of Function Calls")
plt.legend()
plt.grid(True)
plt.show()
```

Output :


Execution Time Comparison
Factorial (Recursive)
Fibonacci Recursive
Fibonacci DP
Execution Time (seconds)
0.000
0.002
0.004
0.006
0.008
6
8
10
12
14
16
18
20
Input Size (n)


Function Calls Comparison
Factorial Calls
Fibonacci Recursive Calls
Fibonacci DP Calls
Number of Function Calls
0
5000
10000
15000
20000
6
8
10
12
14
16
18
20
Input Size (n)

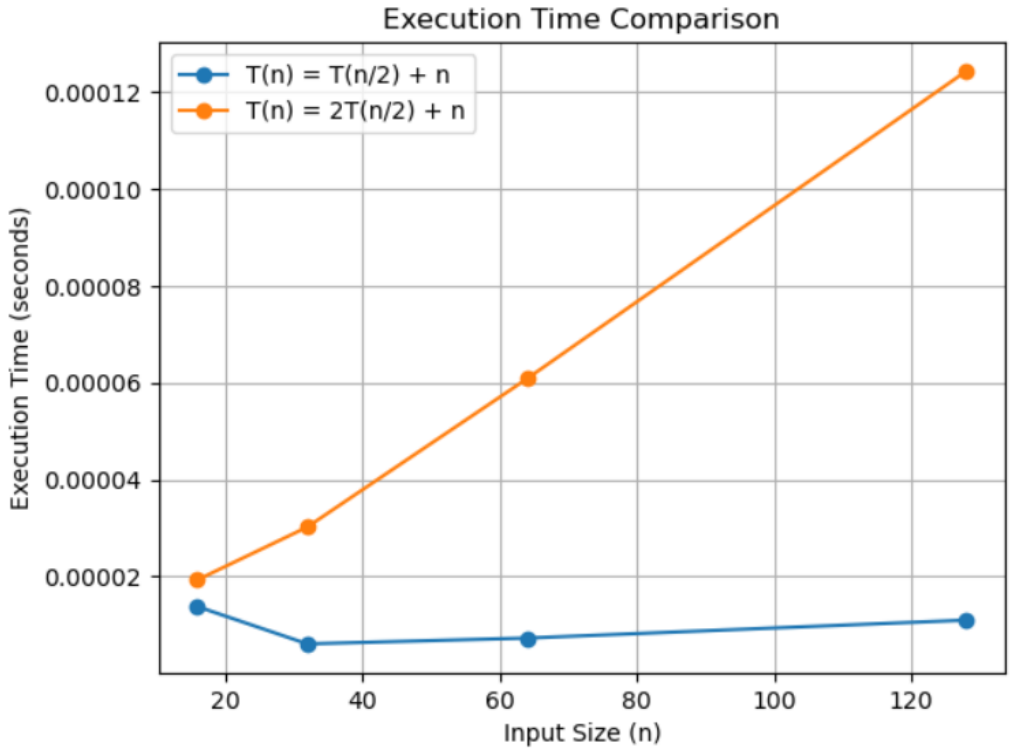
TASK 4: SOLVING RECURRENCES THROUGH CODE

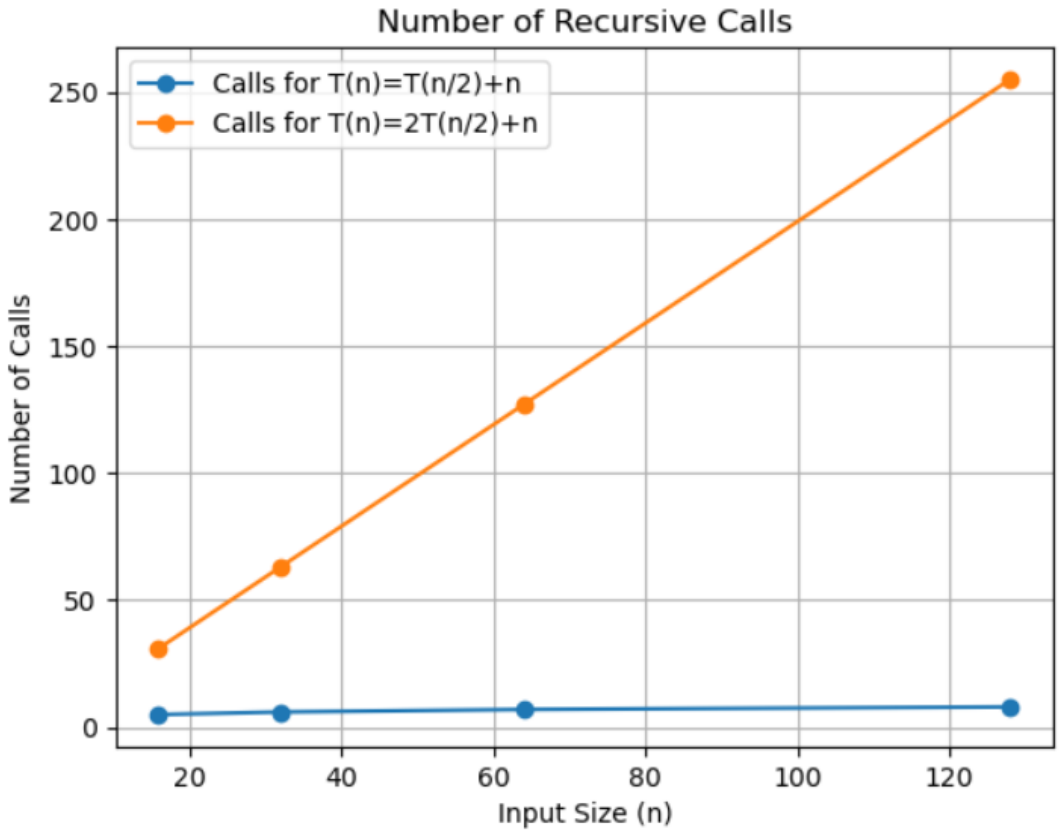
Code :

```
import time
import matplotlib.pyplot as plt

# Recurrence 1: T(n)=T(n/2)+n
calls_r1=0
def recurrence1(n):
    global calls_r1
    calls_r1+=1
    if n<=1:
        return 1
    for i in range(n):
        pass
    return recurrence1(n//2)

# Recurrence 2: T(n)=2T(n/2)+n
calls_r2=0
def recurrence2(n):
    global calls_r2
    calls_r2+=1
    if n<=1:
        return 1
    for i in range(n):
        pass
    return recurrence2(n//2)+recurrence2(n//2)

# Input sizes
n_values = [16,32,64,128]
time_r1=[]
time_r2=[]
calls_list_r1=[]
calls_list_r2=[]
for n in n_values:
```

```
    # Recurrence 1
    calls_r1=0
    start=time.perf_counter()
    recurrence1(n)
    time_r1.append(time.perf_counter()-start)
    calls_list_r1.append(calls_r1)

    # Recurrence 2
    calls_r2=0
    start=time.perf_counter()
    recurrence2(n)
    time_r2.append(time.perf_counter()-start)
    calls_list_r2.append(calls_r2)

# Graph 1: Execution Time
plt.figure()
plt.plot(n_values,time_r1,marker='o',label="T(n) = T(n/2) + n")
plt.plot(n_values,time_r2,marker='o',label="T(n) = 2T(n/2) + n")
plt.title("Execution Time Comparison")
plt.xlabel("Input Size (n)")
plt.ylabel("Execution Time (seconds)")
plt.legend()
plt.grid(True)
plt.show()
# Graph 2: Recursive Calls
plt.figure()
plt.plot(n_values, calls_list_r1,marker='o',label="Calls for T(n)=T(n/2)+n")
plt.plot(n_values, calls_list_r2,marker='o',label="Calls for T(n)=2T(n/2)+n")
plt.title("Number of Recursive Calls")
plt.xlabel("Input Size (n)")
plt.ylabel("Number of Calls")
plt.legend()
plt.grid(True)
plt.show()
```

Output :


Execution Time Comparison
T(n) = T(n/2) + n
T(n) = 2T(n/2) + n
Execution Time (seconds)
0.00012
0.00010
0.00008
0.00006
0.00004
0.00002
20
40
60
80
100
120
Input Size (n)


Number of Recursive Calls
Calls for T(n)=T(n/2)+n
Calls for T(n)=2T(n/2)+n
Number of Calls
250
200
150
100
50
0
20
40
60
80
100
120
Input Size (n)

Observations :

1. The recurrence **T(n)=T(n/2)+n** shows linear growth because each recursive level performs work proportional to n while the problem size halves.
2. The recurrence **T(n)=2T(n/2)+n** generates two recursive calls at each level, increasing the number of calls significantly.
3. Experimental results confirm the theoretical complexities O(n) and O(n log n) respectively.